

Solutions – Week 2: Basic Python Syntax

Complete solutions with explanations, alternatives, and teaching notes for all four practice problems.

Problem 1 – Temperature Converter

Problem Statement

Write a script that converts a temperature from Celsius to Fahrenheit using the formula $F = (C \times 9/5) + 32$. Use a variable to store the Celsius value. Print both values clearly labelled.

Full Solution

```
# Temperature Converter: Celsius to Fahrenheit
celsius = 100
fahrenheit = (celsius * 9/5) + 32
print(str(celsius) + " degrees Celsius = " + str(fahrenheit) + " degrees Fahrenheit")
```

Expected Output

100 degrees Celsius = 212.0 degrees Fahrenheit

Step-by-Step Explanation

Step	Code	What Happens
1	<code>celsius = 100</code>	Stores the integer <code>100</code> in the variable <code>celsius</code>
2	<code>celsius * 9/5</code>	<code>100 * 9 = 900</code> ; then <code>900 / 5 = 180.0</code> (true division → float)
3	<code>180.0 + 32</code>	<code>212.0</code> – the result is a float because step 2 produced a float
4	<code>fahrenheit = 212.0</code>	Stores the computed result

Step	Code	What Happens
5	<code>str(celsius)</code>	Converts integer <code>100</code> to string <code>"100"</code> for concatenation
6	<code>str(fahrenheit)</code>	Converts float <code>212.0</code> to string <code>"212.0"</code> for concatenation
7	<code>print(...)</code>	Outputs the fully assembled string

Alternative – More Pythonic Solution

```
celsius = 100
fahrenheit = (celsius * 9/5) + 32
# f-strings are cleaner and avoid manual str() conversion
print(f"{celsius} degrees Celsius = {fahrenheit} degrees Fahrenheit")
```

F-strings (formatted string literals) embed expressions directly inside `{}`. Python evaluates each expression and converts it to a string automatically – no `str()` needed.

Common Mistakes for This Problem

✗ Using integer division in the formula

```
fahrenheit = (celsius * 9//5) + 32    # 9//5 = 1, not 1.8 – wrong result!
```

✗ Forgetting `str()` when using `+` concatenation

```
print("Result: " + fahrenheit)    # TypeError – can't add str and float
```

Key Concepts Demonstrated

- Arithmetic with mixed operator types (`*` and `/`)
- Float result from true division
- Type conversion with `str()` for string output

Real-World Application

This exact pattern appears in weather apps, IoT sensor dashboards, and scientific data pipelines – sensor data often arrives in one unit and must be converted for display or storage.

Problem 2 – String Analysis

Problem Statement

Given `sentence = "automating the world with python"` , write code to: (a) print it in uppercase, (b) print its length, (c) print only the first word, (d) print it reversed using slicing `[::-1]` .

Full Solution

```
sentence = "automating the world with python"

# (a) Uppercase
print(sentence.upper())

# (b) Length
print(len(sentence))

# (c) First word
print(sentence.split()[0])    # split() breaks on spaces

# (d) Reversed
print(sentence[::-1])
```

Expected Output

```
AUTOMATING THE WORLD WITH PYTHON
32
automating
nohtyp htiw dlrow eht gnitamotua
```

Step-by-Step Explanation

Part	Code	What Happens
(a)	<code>sentence.upper()</code>	Returns a new string with every character converted to uppercase

Part	Code	What Happens
(b)	<code>len(sentence)</code>	Counts all 32 characters including the spaces
(c)	<code>sentence.split()</code>	Splits the string on whitespace, producing <code>['automating', 'the', 'world', 'with', 'python']</code>
(c)	<code>[0]</code>	Accesses the first element of the resulting list: <code>'automating'</code>
(d)	<code>sentence[::-1]</code>	Slice with <code>step = -1</code> traverses the string backwards, reversing it character by character

Alternative – More Explicit Approaches

```
# Alternative for (c): split and unpack
words = sentence.split()
first_word = words[0]
print(first_word)
```

```
# Alternative for (d): using reversed() and join
print("".join(reversed(sentence)))
```

Common Mistakes for This Problem

✗ Using indexing instead of `split()` for the first word

```
print(sentence[0:10])    # Hard-coded – breaks if the first word changes l
```

✗ Expecting `[::-1]` to reverse words rather than characters

```
# sentence[::-1] reverses CHARACTERS: "nohtyp htiw..."
# To reverse WORDS: " ".join(sentence.split()[::-1])
```

Key Concepts Demonstrated

- String method chaining (`split()` + indexing)
- Extended slice notation `[start:stop:step]`
- `len()` counting all characters including spaces

Real-World Application

String reversal, word extraction, and case conversion are used in text normalization (preparing data for NLP models), username validation systems, and log file parsers.

Problem 3 – Paycheck Calculator

Problem Statement

Create variables for employee name, hourly rate, and hours worked. Calculate gross pay, taxes (20% flat rate), and net pay. Print a formatted pay stub.

Full Solution

```
# Employee information
employee_name = "John Smith"
hourly_rate = 18.50
hours_worked = 40

# Calculations
gross_pay = hourly_rate * hours_worked
tax = gross_pay * 0.20
net_pay = gross_pay - tax

# Formatted pay stub
print("===== PAY STUB =====")
print("Employee:      " + employee_name)
print("Hourly Rate:    $" + str(hourly_rate))
print("Hours Worked:   " + str(hours_worked))
print("-----")
print("Gross Pay:      $" + str(gross_pay))
print("Tax (20%):      $" + str(tax))
print("Net Pay:        $" + str(net_pay))
print("=====")
```

Expected Output

```
===== PAY STUB =====
Employee:      John Smith
Hourly Rate:   $18.5
Hours Worked:  40
-----
Gross Pay:     $740.0
Tax (20%):     $148.0
Net Pay:       $592.0
=====
```

Step-by-Step Explanation

Step	Code	Result
1	<code>hourly_rate * hours_worked</code>	$18.50 \times 40 = 740.0$ (float $\times$ int $\rightarrow$ float)
2	<code>gross_pay * 0.20</code>	$740.0 \times 0.20 = 148.0$
3	<code>gross_pay - tax</code>	$740.0 - 148.0 = 592.0$
4	<code>str(gross_pay)</code>	"740.0" – converted for string concatenation

Alternative – Using f-strings and round()

```
employee_name = "John Smith"
hourly_rate = 18.50
hours_worked = 40

gross_pay = hourly_rate * hours_worked
tax = gross_pay * 0.20
net_pay = gross_pay - tax

print(f"{'=' * 30}")
print(f"Employee:      {employee_name}")
print(f"Hourly Rate:   ${hourly_rate:.2f}")
print(f"Hours Worked:  {hours_worked}")
print(f"{'-' * 30}")
print(f"Gross Pay:     ${gross_pay:.2f}")
print(f"Tax (20%):      ${tax:.2f}")
print(f"Net Pay:        ${net_pay:.2f}")
print(f"{'=' * 30}")
```

The `:.2f` format specifier rounds floats to 2 decimal places for proper currency display.

Common Mistakes for This Problem

✗ Adding rate and hours instead of multiplying

```
gross_pay = hourly_rate + hours_worked    # Logical error – no TypeError,
```

✗ Confusing tax with net pay

```
net_pay = gross_pay * 0.20    # This is the tax amount, not net pay
```

Key Concepts Demonstrated

- Float arithmetic (`float * int → float`)
- Chained calculations using intermediate variables
- Type conversion for formatted output

Real-World Application

Payroll systems, invoicing tools, and financial reporting scripts use exactly this pattern – reading employee records, computing deductions, and formatting output for human-readable reports.

Problem 4 – Comparison Operators

Problem Statement

Create `x = 15` and `y = 27`. Print the result of all six comparison operators with clear labels.

Full Solution

```
x = 15
y = 27

print("x == y :", x == y)
```

```
print("x != y :", x != y)
print("x > y  :", x > y)
print("x < y  :", x < y)
print("x >= y :", x >= y)
print("x <= y :", x <= y)
```

Expected Output

```
x == y : False
x != y : True
x > y  : False
x < y  : True
x >= y : False
x <= y : True
```

Step-by-Step Explanation

Expression	Evaluation	Result
<code>x == y</code>	Is <code>15</code> equal to <code>27</code> ?	<code>False</code>
<code>x != y</code>	Is <code>15</code> not equal to <code>27</code> ?	<code>True</code>
<code>x > y</code>	Is <code>15</code> greater than <code>27</code> ?	<code>False</code>
<code>x < y</code>	Is <code>15</code> less than <code>27</code> ?	<code>True</code>
<code>x >= y</code>	Is <code>15</code> greater than or equal to <code>27</code> ?	<code>False</code>
<code>x <= y</code>	Is <code>15</code> less than or equal to <code>27</code> ?	<code>True</code>

When a comma separates arguments in `print()` , Python automatically inserts a space and converts each argument to a string – no `str()` needed for `bool` values.

Alternative – Storing Results in Variables

```
x = 15
y = 27

results = {
    "x == y": x == y,
    "x != y": x != y,
```



```
"x > y ": x > y,  
"x < y ": x < y,  
"x >= y": x >= y,  
"x <= y": x <= y,  
}  
  
for label, result in results.items():  
    print(f"{label} : {result}")
```

This approach uses a dictionary to organise the data and a loop to print it – a preview of Week 5 (Data Structures) and Week 3 (Control Flow).

Common Mistakes for This Problem

✗ Using `=` instead of `==`

```
print(x = y)      # SyntaxError  
if x = y:         # SyntaxError
```

✗ Confusing `>=` with `>`

```
# x >= y means True if x > y OR x == y  
# x > y means True ONLY if x is strictly greater  
print(5 >= 5)     # True  
print(5 > 5)      # False
```

Key Concepts Demonstrated

- All six Python comparison operators
- `bool` as a return type from comparisons
- Passing non-string arguments to `print()` using commas

Real-World Application

Comparison operators are the foundation of every decision in a program – access control checks (`user_level >= required_level`), data validation (`temperature < max_threshold`), and loop conditions (`count <= total`).

Cross References

- **Concept files:** [Lab_2_1_Variables_and_DataTypes.md], [Lab_2_2_Operators.md], [Lab_2_3_StringOps_TypeConversion.md]
- **Problems:** [Practice_Problems.md]
- **Challenge:** [Challenge_Lab.md]
- **Quick reference:** [Revision_Sheet.md]